Analisi semantica

1 Gestione dello scope

Il linguaggio **pike** adotta un modello a **single scope**. Non esiste distinzione tra variabili locali e globali.

- Tutte le variabili, incluse quelle definite all'interno delle funzioni o passate come argomenti, vengono registrate in un'unica tabella dei simboli globale.
- Una variabile definita dentro una funzione è visibile anche all'esterno dopo l'esecuzione della funzione stessa (e viceversa).
- Non è possibile avere due funzioni con lo stesso nome (o una variabile con lo stesso nome di una funzione) poiché condividono lo stesso scope.

2 Inferenza di tipo

Il linguaggio è tipizzato staticamente nelle firme delle funzioni, ma utilizza l'inferenza di tipo per le variabili locali tramite assegnazione.

2.1 Variabili

Il tipo di una variabile non viene dichiarato esplicitamente. Viene inferito dall'espressione a destra dell'operatore di assegnazione `:=`.

- Se `x := 5`, viene inferito che `x` è di tipo `num`.
- Se `s := "testo"`, viene inferito che `s` è di tipo `str`.

2.2 Propagazione

Il tipo inferito si propaga attraverso le espressioni con i numeri.

Se `a` è `num` e `b` è `num`, allora `c := a + b` inferisce che `c` è `num`.

2.3 Funzioni

- I tipi degli argomenti sono espliciti (`a: num`).
- Il tipo di ritorno è esplicito (`% num`). Può essere omesso, in questo caso restituisce `0.0` (rappresentando una sorta di `void`).

3 Type checking

Il controllo dei tipi deve garantire la coerenza delle operazioni prima o durante l'esecuzione.

Queste sono le regole principali.

3.1 Operazioni aritmetiche e logiche

- **Operatori binari (`+`, `-`, `*`, `/`)**: Entrambi gli operandi devono essere rigorosamente di tipo `num`. Non è supportata la concatenazione di stringhe con `+`.
- **Operatori di confronto (`<`, `>`)**: Entrambi gli operandi devono essere di tipo `num`.

3.2 Assegnazioni e ridefinizioni

Una volta che a una variabile è stato assegnato un tipo (es. `num`), un successivo tentativo di assegnare un valore di tipo diverso (es. `str`) dovrebbe generare un errore.

3.3 Chiamate a funzione

- Il numero di argomenti passati nella chiamata deve coincidere esattamente con il numero di argomenti nella definizione della funzione.
- Il tipo di ogni espressione passata come parametro deve coincidere con il tipo dichiarato nella firma della funzione (es. non si può passare una `str` dove è atteso un `num`).

3.4 Controllo di flusso

- L'espressione condizionale all'interno di `if (...)` e la condizione di terminazione nel `for (...)` devono valutare un tipo compatibile con la logica booleana. Dato che il linguaggio ha solo `num` e `str`, si assume che:
  - La condizione deve derivare da un confronto (che produce un valore logico interno) oppure essere un `num` (dove `0` è *false* e qualsiasi altro valore è *true*).
  - L'uso di una stringa come condizione genera un errore di tipo.

3.5 Valore di ritorno

- Se una funzione dichiara un tipo di ritorno (es. `% num`), l'espressione associata allo statement `ret` deve essere di quel tipo.

- Se una funzione non dichiara un tipo di ritorno, non può restituire valori (o `ret` deve essere vuoto/assente).